


# CLEANALERT: SMART WASTE COLLECTION NOTIFICATION AND COMMUNITY REWARD SYSTEM

**Devendra singh - 24619210**

**Sourabh kumar - 24619253**

## MINI PROJECT REPORT

Submitted to the

**SCHOOL OF ENGINEERING**

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**

in Partial fulfillment of the requirements for the award of the degree of

**BACHELOR OF TECHNOLOGY CSE**

**IN**

**ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**




**VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)**

**CHENNAI – 600 117**

**OCTOBER 2026**

# BONAFIDE CERTIFICATE

Certified that this project report entitled "**CLEANALERT: SMART WASTE COLLECTION NOTIFICATION AND COMMUNITY REWARD SYSTEM**" is the bonafide work of **DEVENDRA SINGH - (24619210)** and **SOURABH KUMAR - (24619253)**, carried out under the supervision of the project guide. The final names, register numbers, and guide details are to be completed and verified by the department before submission.

SIGNATURE OF THE HOD

**Dr. K. Ulaga Priya M.E., Ph.D**
Associate Professor & Head
Of Department AI&ML
School of Engineering
VISTAS
Chennai

SIGNATURE OF THE SUPERVIOR

**S J Kavitha, M.E.,(Ph.d)**
Assistant Professor
Department of AI&ML
School of Engineering
VISTAS
Chennai

Submitted for the Project Work and Viva - Voce examination held on…… at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

**INTERNAL EXAMINER** **EXTERNAL EXAMINER**

## ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support I received from the positive minds around me in making this venture a successful one. The infrastructural support with all kinds of lab facilities have been a motivating factor in this completion of project work, all because of FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH., with great pleasure that I take this opportunity to thank him. I am grateful to our **HEAD OF THE DEPARTMENT, Dr. K. Dr.Ulaga Priya,** for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day. I extend our warmest thanks to my **INTERNAL GUIDE, S J KAVITHA Ass.Prof/AI&ML,** who has patiently guided and provided us with invaluable advice and help. I would like to express my greatest gratitude to our Staff members, Librarian, and Non-teaching staff members of Computer Centre, who have helped and supported me throughout. Last but not the Least, I wish to thank my parents and my friends for their individual support and interest who has inspired me and encouraged me to go in my own way, without them I would be unable to complete this project.

**DEVENDRA SINGH - (24619210)**

**SOURABH KUMAR - (24619253)**

| CHAPTER NO. | TABLE OF CONTENT | PAGE NO. |
|---|---|---|
| | **ABSTRACT** | i |
| | **LIST OF FIGURES** | ii |
| | **ABBREVIATIONS** | iii |
| 1 | **INTRODUCTION** | 1 |
| | 1.1 Overview of the project | 01 |
| | 1.2 Scope and objective | 02 |
| **2** | **LITERATURE REVIEW** | 02 |
| | 2.1 Literature Overview | 02 |
| | 2.2 Literature Survey | 03 |
| **3** | **SYSTEM DESIGN** | 04 |
| | 3.1 Existing System | 04 |
| | 3.2 Proposed System | 04 |
| | 3.3 System Architecture | 04 |
| | 3.4 Hardware Requirements | 06 |
| | 3.5 Software Requirements | 06 |
| **4** | **IMPLEMENTATION** | 06 |

| CHAPTER NO. | TABLE OF CONTENT | PAGE NO. |
|---|---|---|
| | 4.1 Methodology | 06 |
| | 4.2 Implementation | 07 |
| | 4.2.1 Authentication and Registration Module | 07 |
| | 4.2.2 Resident Module | 10 |
| | 4.2.3 Collector Module | 11 |
| | 4.2.4 Administrator Module | 11 |
| | 4.2.5 Location and Notification Module | 11 |
| | 4.2.6 Rewards and Submission Review | 14 |
| **5** | **CONCLUSION AND FUTURE WORK** | 15 |
| | 5.1 Conclusion | 15 |
| | 5.2 Future Work | 15 |
| | 5.3 Testing and Results | 15 |
| | 5.4 Source Code | 16 |
| | **REFERENCES** | |

# ABSTRACT

CleanAlert is an Android-based smart waste management application designed to improve communication between residents, waste collectors, and administrators. In many localities, residents may not know when a collection vehicle is approaching, while collection teams and administrators may lack a shared digital view of submissions, routes, and completed work. CleanAlert brings these workflows together through role-based screens, photo-based waste submissions, route assignment, collection history, and a community reward mechanism. The resident workflow supports account registration and login, selecting a waste category, attaching a photograph, viewing submission history, and checking points and rewards. The collector workflow provides a dashboard, availability status, assigned route information, a map view, and collection history. The administrator workflow includes user and collector management, route assignment, submission review, reward catalogue management, and reports. The project is designed around Kotlin and Jetpack Compose for the Android interface, Firebase Authentication for account access, Cloud Firestore for application data, Firebase Storage for submitted images, Firebase Cloud Messaging for notifications, and Google Maps and Android location services for map-related functionality. The rewards workflow is intended to encourage verified participation in responsible waste disposal. This report presents the project overview, related concepts, system design, requirements, methodology, implementation modules, testing plan, and future scope. The screenshots demonstrate the planned interface structure; however, live GPS updates, notification delivery, backend security, and reward accounting must be validated through end-to-end testing before operational claims can be made.



**Keywords**: Smart waste management, Android, GPS, Firebase, Google Maps, waste collection, community rewards, notifications.

ISBN No.: 978-93-344-7370-4

| FIGURE NO | TITLE | PAGE NO. |
|---|---|---|
| Figure 1 | CleanAlert high-level system architecture | 05 |
| Figure 2 | Resident sign-in screen | 08 |
| Figure 3 | Resident and collector registration screens | 09 |
| Figure 4 | Administrator dashboard | 10 |
| Figure 5 | User and collector management screen | 12 |
| Figure 6 | Collector route assignment screen | 13 |

# ABBREVIATIONS

| ABBREVIATION | FULL FORM |
|---|---|
| AI | Artificial Intelligence |
| API | Application Programming Interface |
| FCM | Firebase Cloud Messaging |
| GPS | Global Positioning System |
| GUI | Graphical User Interface |
| IoT | Internet of Things |
| SDK | Software Development Kit |
| UI | User Interface |
| UX | User Experience |

# CHAPTER 1. INTRODUCTION

## 1.1 OVERVIEW OF THE PROJECT

Waste collection is an important public service, but residents do not always receive timely information about collection schedules or vehicle progress. When collection timing is uncertain, waste may remain outside homes or on streets for longer than necessary. Collection teams also need a clear way to view assigned routes and record completed work. A mobile application can reduce this communication gap by making collection information and resident participation easier to manage.

CleanAlert is designed as a shared platform for three user roles: residents, collectors, and administrators. Residents can submit photographs of waste and track their submissions and rewards. Collectors can view assigned route information and collection history. Administrators can manage users, assign routes, review submissions, configure rewards, and inspect operational information.The system combines a mobile-first interface with location-aware functionality and a community reward concept. Its goal is to support better coordination and responsible waste handling. It is intended to complement collection services, not replace municipal planning or field operations.

**Problem Statement**

Traditional collection workflows may rely on fixed schedules, local announcements, or direct communication. These methods can provide limited visibility into vehicle progress and may not offer a central record of resident submissions, route assignments, or verified actions. The problem addressed by CleanAlert is the lack of a simple shared digital workflow that connects residents, collectors, and administrators.

**Objectives of the Project**

- Develop a mobile application with distinct resident, collector, and administrator workflows.
- Provide account creation and sign-in screens for supported user roles.
- Allow residents to submit waste photographs with a selected waste category.
- Maintain a history of submissions and show reward points and available rewards.
- Provide collectors with assigned route information, a map screen, and collection history.
- Enable administrators to manage users, routes, submissions, rewards, and reports.
- Design the application so that cloud data and push notifications can be integrated securely.

**Scope of the Project**

- Resident account, dashboard, waste submission, history, and rewards interfaces.
- Collector dashboard, availability status, route/map view, and collection history.
- Administrator dashboard, user/collector management, route assignment, submission review, reward catalogue, and reports.
- Cloud storage for submission records and photographs, subject to configured backend rules.
- Location-aware map functionality and future collection-arrival notifications.

### Importance and Applications

A common application can make waste-related information easier to access and reduce fragmented communication. Photo submissions can provide a record for review, while rewards can encourage residents to participate. The system may be useful in residential communities, apartment complexes, campus environments, and local collection pilots, provided that routes and operating responsibilities are agreed with the relevant service provider.

## 1.2 SCOPE AND OBJECTIVES

### General Objective

The general objective is to design a mobile-based waste management support system that connects residents, collectors, and administrators through waste submissions, route visibility, and a verified community reward workflow.

### Specific Objectives

1. Create role-aware account and dashboard screens.
2. Capture or select a photograph and attach it to a waste submission.
3. Store submission metadata and review status in the application backend.
4. Display user activity, points, and reward catalogue information.
5. Allow administrators to assign collection routes to registered collectors.
6. Provide a map-based view for route awareness and collection tasks.
7. Define test cases for account access, uploads, route assignment, rewards, and permissions.

### Expected Outcomes

The expected outcome is a demonstrable prototype covering the main screens and role-specific workflows. Actual benefits such as reduced pickup delays or improved recycling participation require a field pilot and measured evaluation.

- The application aims to make waste collection activities more transparent and organised for all users.
- It provides a central platform where waste-related information can be recorded and accessed.
- Photographs submitted by residents can help administrators review reported waste and maintain records.
- The collector module supports better coordination by displaying assigned routes and collection history.
- Role-based access helps ensure that users are directed to functions relevant to their responsibilities.
- Cloud-based services can support the storage and synchronisation of application data.
  Location-related features can improve awareness of collection routes and nearby service activities.
- The reward system is intended to encourage residents to participate in responsible waste disposal.

# CHAPTER 2. LITERATURE REVIEW

## 2.1 LITERATURE OVERVIEW

Smart waste management combines information systems, mobile applications, geographic information, and operational data to improve the coordination of waste collection. Traditional collection systems commonly use fixed schedules and manual coordination. These approaches are simple, but they may not provide residents with timely updates or give administrators a unified view of collection activity.

Mobile applications can improve communication by allowing residents to submit reports, view status updates, and receive notifications. Geographic information systems and GPS-enabled devices can provide route context and support tracking, while cloud databases can store submissions and operational records. Reward systems are another approach used to encourage participation, although their effectiveness depends on transparent rules and verification.

### 2.1.1 Role of Digital Technology in Waste Management

Digital technology can improve the coordination of waste collection services.
Mobile applications provide a convenient way for residents to share waste-related information.
Digital records help maintain information about reported waste and collection activities.
Cloud databases allow authorised users to access updated information from different devices.
Photographic evidence can support the review of waste submissions by administrators.
These features can help reduce dependence on manual records and informal communication.

### 2.1.2 Importance of GPS and Mobile Applications

GPS technology provides location information that can support route awareness.
Map interfaces help collectors identify assigned locations and understand route details.
Mobile notifications can inform users about important updates when properly configured.
However, accurate location tracking depends on device permissions and network availability.
Battery usage and user privacy must also be considered when collecting location data.

### 2.1.3 Community Participation and Reward Systems

Public participation is an important part of effective waste management.
Residents can contribute by reporting waste and following responsible disposal practices.
A point-based reward system can encourage users to participate regularly.
Submission verification is necessary to prevent duplicate or invalid reward claims.
Clear reward rules can help maintain fairness and transparency among participants..

## 2.2 LITERATURE SURVEY

**Traditional Waste Collection**

Conventional collection often follows predetermined routes and schedules. Residents may depend on local announcements or personal experience to estimate arrival times. If routes change or vehicles are delayed, information may not reach every household. A digital platform can provide a central place for route assignments and service updates.

**Mobile-Based Reporting Systems**

Mobile reporting applications allow users to share photographs, locations, and descriptions of local issues. For waste management, these features can help record overflowing bins, missed pickups, or waste categories. CleanAlert applies a related approach by allowing residents to submit photographs and review the status of their submissions.

**GPS and Route Visibility**

Location services and map interfaces can help users understand where a collection task is located and can support route navigation. Reliable live tracking requires location permissions, an update mechanism, network connectivity, and backend logic. Displaying a map alone should not be treated as proof that a vehicle is being tracked in real time.

**Cloud Services and Notifications**

Cloud databases support synchronised access to records across multiple user roles. File storage can hold photographs, while push notification services can notify users about relevant events. These services require carefully configured access rules and error handling to protect user data and maintain consistent records.

**Community Rewards**

Point-based incentives can make participation more visible and encourage repeated engagement. A reliable reward system must define which actions qualify, who verifies them, and how duplicate or fraudulent claims are prevented. In CleanAlert, the recommended workflow is to award points only after a submission has been reviewed and approved.

**Research Gap and Project Motivation**

The project focuses on combining resident submissions, collector route assignment, administrative review, and rewards in one mobile workflow. The intended contribution is integration and usability rather than a claim of a new routing or machine-learning algorithm. A future evaluation should compare actual collection awareness, response time, and participation before and after a field pilot.

This chapter summarises the technical concepts relevant to the project. Formal submission should add institution-approved, verifiable research references if the department requires a detailed literature survey.

# CHAPTER 3. SYSTEM DESIGN

## 3.1 EXISTING SYSTEM

In a conventional workflow, residents may depend on fixed schedules, community announcements, or direct contact with collection workers. Route information and pickup records may be maintained separately, and residents may have little visibility into the status of a reported issue.

**Limitations of the Existing System**

- Residents may not know when the collection vehicle will arrive.
- Route changes and delays may not be communicated consistently.
- Photo evidence and submission status may not be centrally recorded.
- Reward or participation records may be difficult to verify.
- Administrators may need to coordinate users and collectors manually.

## 3.2 PROPOSED SYSTEM

CleanAlert proposes a role-based mobile application that brings resident reporting, collector route information, administrative review, and rewards into a common workflow. Residents can submit photographs and check their activity. Collectors can view assigned routes and collection history. Administrators can manage accounts, assign routes, review submissions, configure rewards, and view available reports.

**Main Features**

- Account registration and login for residents and collectors.
- Photo-based waste submissions with category selection.
- Resident history and reward-point display.
- Collector dashboard, availability status, route information, map, and history.
- Administrator user and collector management.
- Route assignment and editing.
- Submission review and reward catalogue management.
- Reports and operational counts.

## 3.3 SYSTEM ARCHITECTURE

Component responsibilities and interaction

The presentation layer is responsible for collecting user input and displaying the current state of a task. The application logic validates inputs, checks the active user role, and coordinates requests to the backend. Authentication establishes the signed-in identity, while role information determines which functions are available. Database records should be the authoritative source for submission status, route assignment, reward balances, and collection history. Photographs should be stored as files, with their storage references and related metadata recorded in the database.

Failure handling is important because mobile connectivity can be intermittent. The interface should show a clear loading state while a request is in progress, confirm success only after the backend responds, and provide a retry option when an upload or data request fails. Repeated submissions and reward requests should be handled idempotently so that a retry does not create duplicate records or award points twice. The architecture should also keep secrets such as API keys out of source control and use backend security rules to enforce permissions.

The proposed architecture has a mobile presentation layer, application logic, cloud services, and external map/location services. The presentation layer contains screens for each role. Application logic validates actions and applies workflow rules. Firebase services can provide authentication, data storage, image storage, and push notifications. Google Maps and Android location services support geographic presentation.

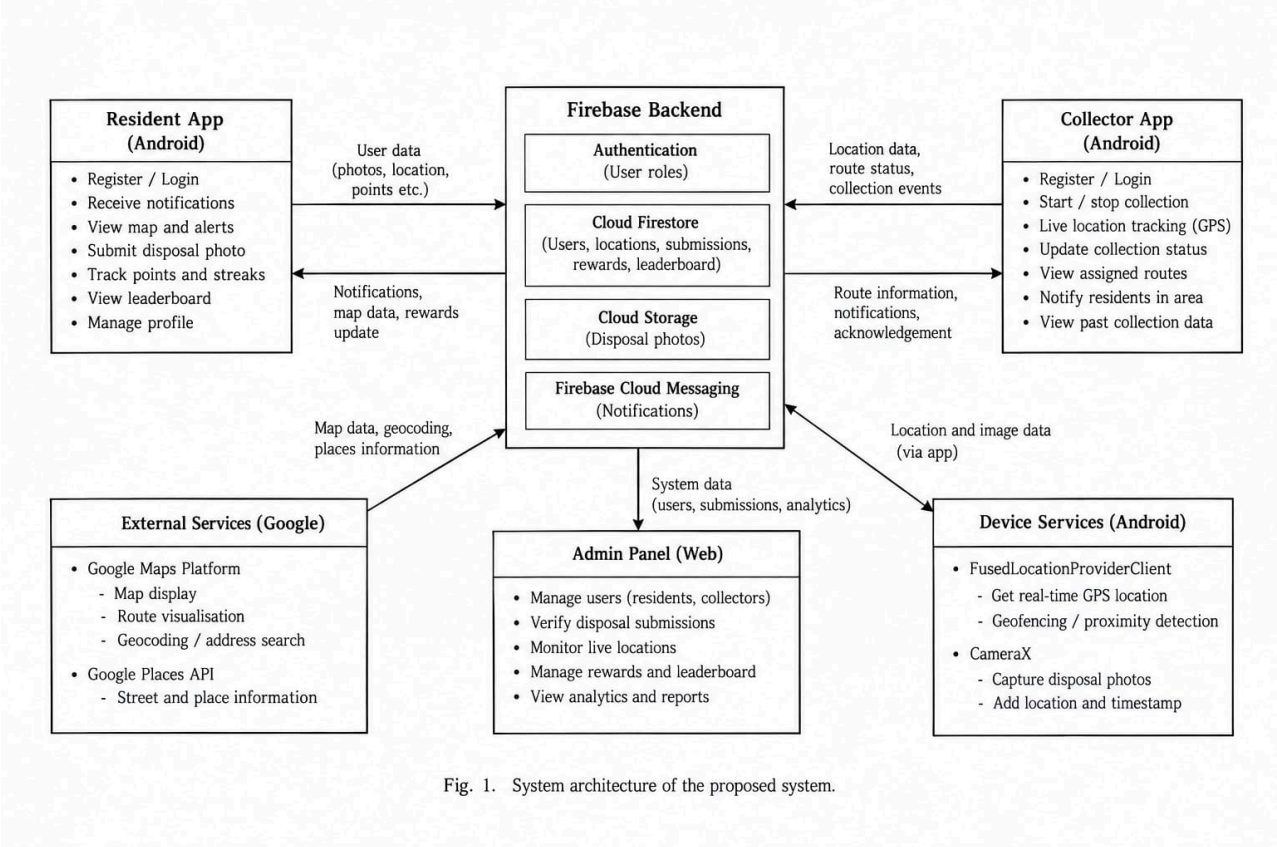



Fig. 1. System architecture of the proposed system.

*Figure 1. High-level system architecture of CleanAlert*

Description: The architecture diagram presents CleanAlert as a mobile application connected to Firebase services and external device services. The resident and collector applications exchange user, submission, route, and collection information with the backend. Firestore stores application records, Cloud Storage holds submitted photographs, and Firebase Cloud Messaging supports event notifications when configured. Google Maps and Android location services provide map and location capabilities. The diagram represents the intended system structure; each cloud service and live update path must be verified in an end-to-end test.

## Data Flow

- The user registers or signs in and enters the relevant role-specific interface.
- A resident selects a waste category and attaches a photograph to a new submission.
- The application stores the submission metadata and image reference through the configured backend.
- An administrator reviews the submission and changes its status.
- Approved submissions may update the resident's points and activity history.
- An administrator assigns a route to a collector, who views the route information in the collector interface.

- Notifications may be sent when the relevant backend events and messaging services are configured.

**Main Data Entities**

- User: user ID, name, email, role, and profile information.
- Submission: submission ID, resident ID, category, photo reference, timestamp, location, and review status.
- Collector route: collector ID, assigned route, route stops, availability, and progress.
- Reward: reward ID, title, description, points required, active status, and redemption record.
- Notification: recipient, event type, related record, delivery status, and timestamp.

## 3.4 HARDWARE REQUIREMENTS

| COMPONENT | PURPOSE |
|---|---|
| Development computer | A computer capable of running Android Studio and an Android emulator. |
| Android device | An Android smartphone for testing registration, image capture, location permissions, and UI behaviour. |
| Internet connection | Required for cloud authentication, database access, image upload, and notifications. |
| GPS/location capability | Required for testing map and location-aware functions. |
| Camera | A phone camera or image picker for attaching photographs to submissions. |

## 3.5 SOFTWARE REQUIREMENTS

Configuration and deployment considerations

The Android project must be built with compatible versions of the Android Gradle Plugin, the Android SDK, and the required Firebase and Google Play services libraries. Firebase must be configured for the correct application package, and the required Authentication, Firestore, Storage, and Cloud Messaging services must be enabled in the project console. The Maps API key should be configured through a local or deployment-specific property rather than hard-coded into source files. Before distribution, the developer should confirm that the API key is restricted to the intended application and that billing and API permissions are configured where required.

| SOFTWARE / SERVICE | PURPOSE |
|---|---|
| Operating System | Windows or Linux supported by Android Studio |
| Development Environment | Android Studio |
| Programming Language | Kotlin |
| UI Framework | Jetpack Compose and Material components |
| Authentication | Firebase Authentication |
| Database | Cloud Firestore |
| Image Storage | Firebase Storage |
| Notifications | Firebase Cloud Messaging |

| | |
|---|---|
| Maps and Location | Google Maps SDK and Android location APIs |
| Version Control | Git, if configured in the project |

# CHAPTER 4. IMPLEMENTATION

## 4.1 METHODOLOGY

Validation and iteration
After each module is implemented, it should be checked independently before being connected to the next module. For example, registration should be tested before role-based navigation, and image upload should be verified before testing the complete submission-review workflow. Integration testing should then follow realistic user journeys across resident, collector, and administrator accounts. Any defect found during testing should be recorded with the steps to reproduce it, the expected behaviour, the observed behaviour, and the result after correction. This approach makes it easier to identify whether a failure comes from the interface, permissions, network connection, or backend configuration.

The project follows a modular development methodology. The workflow starts with requirement analysis, then defines role-specific screens and data entities, integrates cloud services, and validates the user journeys. The implementation is separated into modules so that resident functions, collector operations, administrator controls, and rewards can be developed and tested independently.

**Development Stages**

- Requirement Analysis: identify the needs of residents, collectors, and administrators.
- Interface Design: create the screen flow for registration, dashboards, submissions, routes, rewards, and administration.
- Data Model Design: define records for users, submissions, routes, rewards, and notifications.
- Authentication Integration: implement account creation, sign-in, and role-aware navigation.
- Submission Workflow: implement category selection, photo attachment, and submission history.
- Collector Workflow: implement route display, availability status, and collection history.
- Administrator Workflow: implement user management, route assignment, submission review, rewards, and reports.
- Testing: verify each module and then test complete end-to-end workflows.

## 4.2 IMPLEMENTATION

### 4.2.1 Authentication and Registration Module

The sign-in screen provides email and password fields, a sign-in action, a password recovery entry point, and a Google sign-in option shown in the interface. The registration screen allows the user to select the Resident or Collector role. Collector registration includes a personal ID or licence-number field. Authentication providers and account creation behaviour must be checked against the final Firebase configuration.

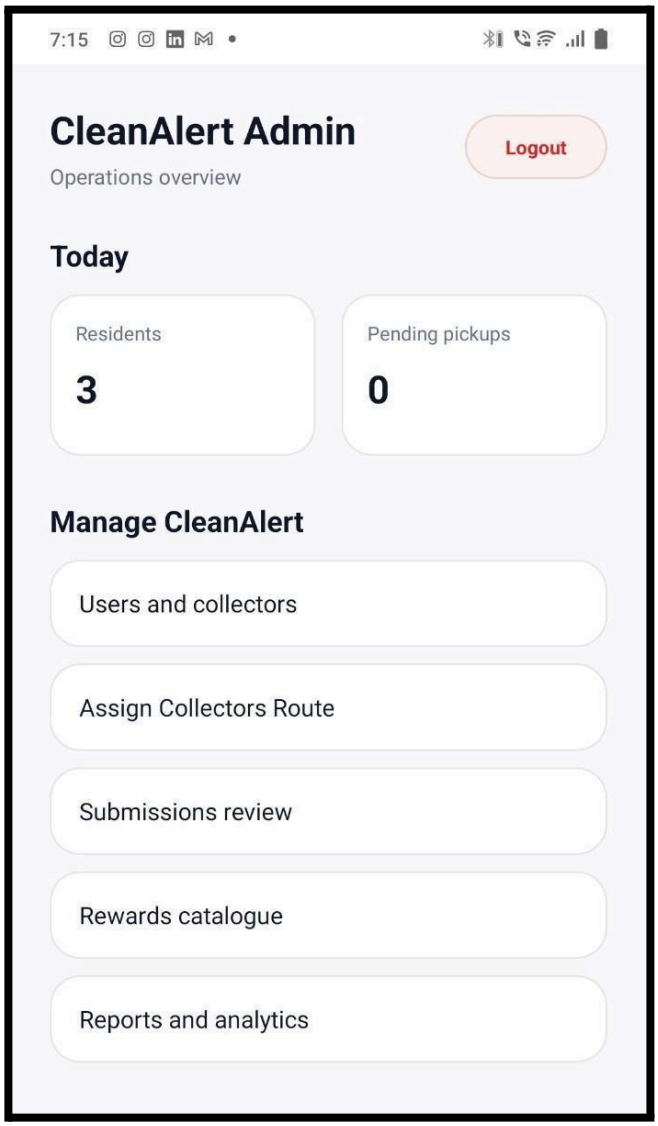



*Figure 2. CleanAlert administrator dashboard*

Description: This dashboard gives the administrator a quick operational overview, including the displayed resident count and pending-pickup count. The menu provides entry points for user and collector management, route assignment, submission review, reward-catalogue management, and reports. These shortcuts organise common administrative tasks in one place. The figures shown are sample interface values and should not be treated as measured operational results.The dashboard also helps administrators identify the main activities that require attention.The user management section supports the organisation of resident and collector accounts.Route assignment options help administrators coordinate collection responsibilities.The submission review section provides a place to examine resident reports.Reward and reporting features support the management of community participation.Overall, the dashboard brings important administrative functions together in a single interface.

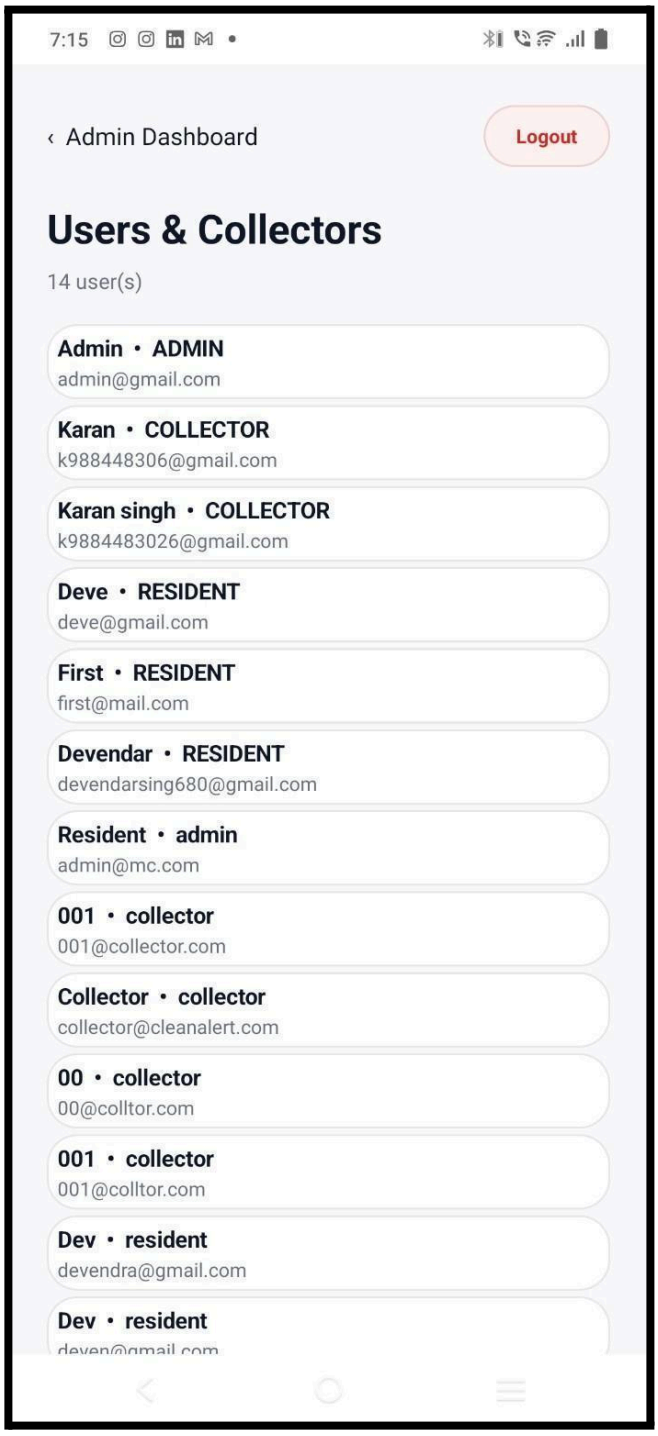



*Figure 3. Users and collectors management screen*

Description: This screen lists registered accounts with their displayed names, email addresses, and role labels. It helps an administrator distinguish residents from collectors and review the accounts registered in the application. In a production deployment, access to this screen should be restricted to authorised administrators, and personal information should be displayed only when required for the task.

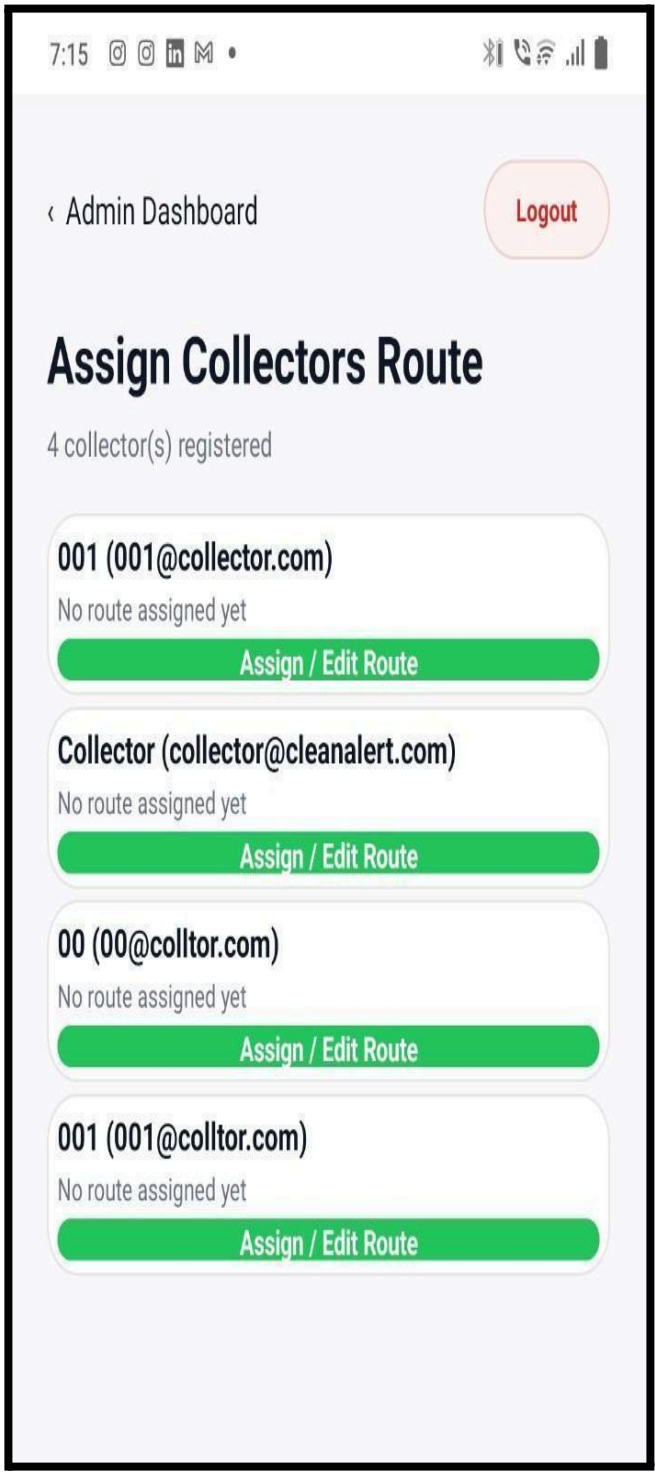



*Figure 4. Assign collectors route screen*

Description: The route-assignment interface lists registered collectors and shows whether a route has been assigned. The Assign/Edit Route control provides the intended entry point for assigning or changing a collector's route. After a route is saved, the application should validate that the selected collector exists and that the assignment is persisted to the backend before presenting it as complete.

### 4.2.2 Resident Module

The resident module is intended to provide the user with a central dashboard for submitting waste, viewing activity history, and checking reward points. A submission flow includes choosing a waste category, attaching an image, and sending the record for review. The application should clearly indicate whether a submission is pending, approved, or rejected.

- Resident profile and account information.
- Waste category selection and photo capture/upload.
- Submission history and status.
- Points balance and reward catalogue.
- Clear feedback for upload failures or missing permissions.

### 4.2.3 Collector Module

Collector workflow and status handling

A collector's dashboard should make the current availability status and assigned collection tasks easy to identify. When a route is changed by an administrator, the collector view should refresh from the stored assignment rather than relying only on a locally cached value. Each collection task should move through clearly defined states, such as pending, assigned, and completed, with each transition limited to the roles authorised to perform it. The application should display a useful message if no route is assigned or if the latest route information cannot be retrieved.

The collector module provides a dashboard showing collection-related information and a route/map entry point. The collector can view assigned route information and collection history. The interface also includes an availability status. Route details should be fetched from the authoritative backend so that an administrator's route changes are reflected consistently.

### 4.2.4 Location and Notification Module

Location privacy and notification reliability

Location access should be requested only when the user opens a feature that needs it, with a clear explanation of why it is required. The application should handle denied permissions and unavailable GPS without crashing or blocking unrelated features. If collector location is shared, the system should define when sharing starts and stops, limit access to appropriate roles, and avoid retaining location data longer than necessary. Push notifications should be tied to a defined backend event and should not be considered delivered merely because a request was initiated; delivery and device receipt should be tested separately.

Google Maps and Android location services are intended to provide map display and location-aware functionality. Live tracking requires a valid location update pipeline, device permissions, network availability, and suitable backend handling. Firebase Cloud Messaging can be used for notifications when a route-related or submission-related event occurs. These functions should be tested on physical devices and should not be described as operational solely because the relevant screens exist.

### 4.2.5 Administrator Module

The administrator dashboard provides access to user and collector management, route assignment, submission review, reward catalogue, and reports. The route assignment screen lists registered collectors and provides an Assign/Edit Route action. User management lists accounts with their displayed roles and email addresses.

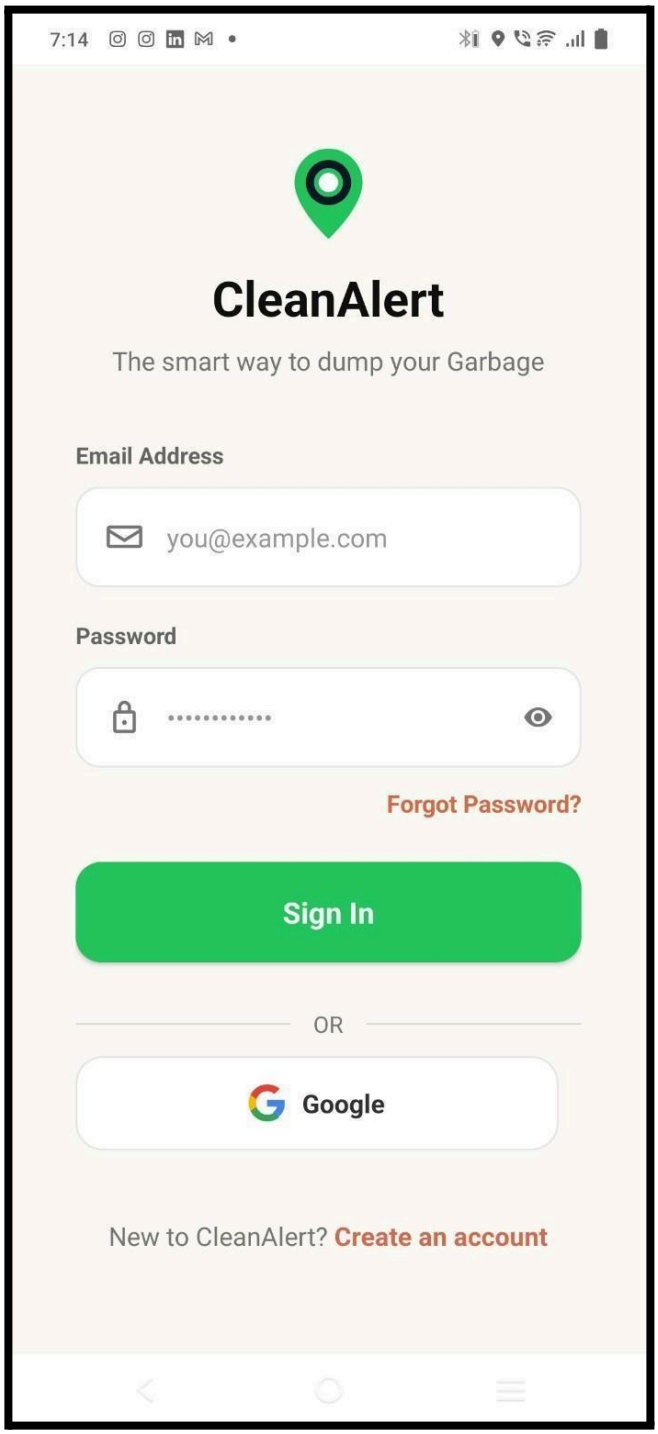



*Figure 5. CleanAlert sign-in screen*

Description: The sign-in screen contains email and password fields, a sign-in button, a password-recovery link, and a Google sign-in option. It also provides a path to account creation for new users. The interface is designed to give users a single entry point before routing them to the appropriate role-based area. Successful authentication and Google sign-in depend on the Firebase and OAuth configuration being correctly enabled.

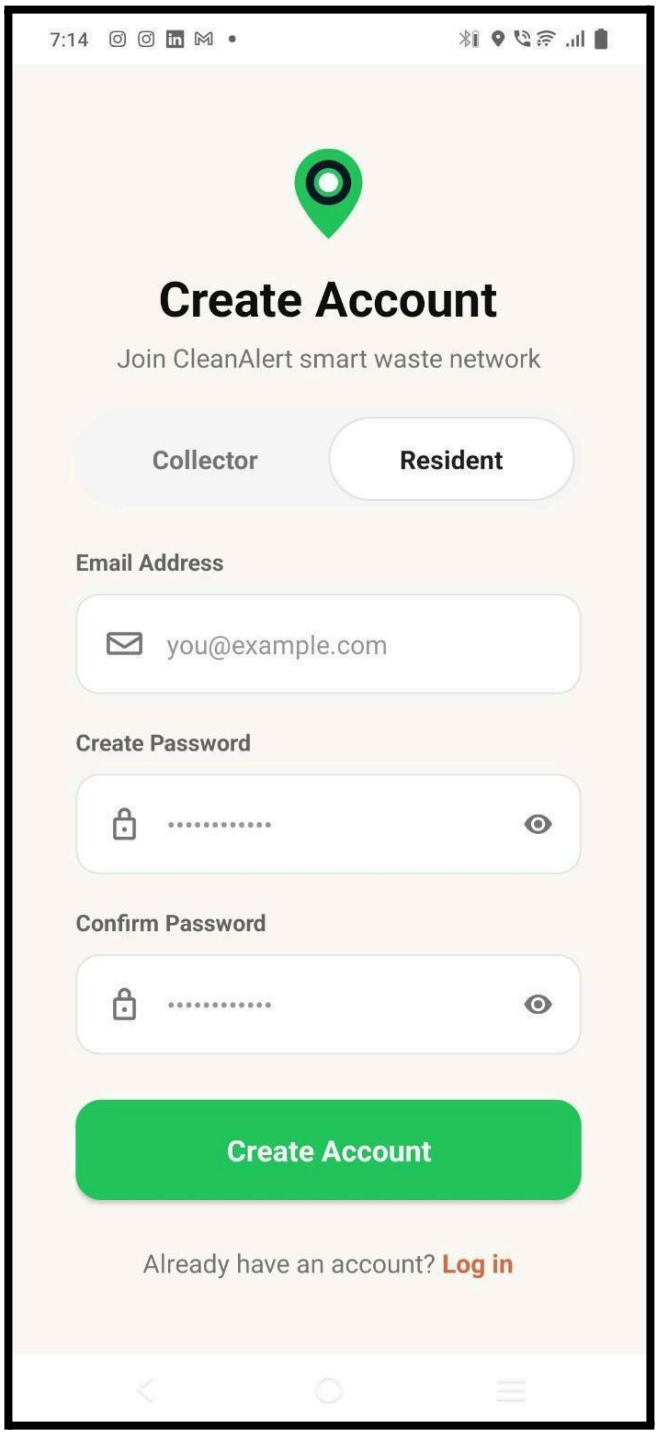



*Figure 6. Resident account registration screen*

Description: The registration screen allows the user to select the Resident role and enter an email address, password, and password confirmation. The form provides a Create Account action and a link back to sign-in. Input validation should reject missing fields, weak passwords, and mismatched passwords, while the backend should create the account and the corresponding resident profile only after successful authentication.

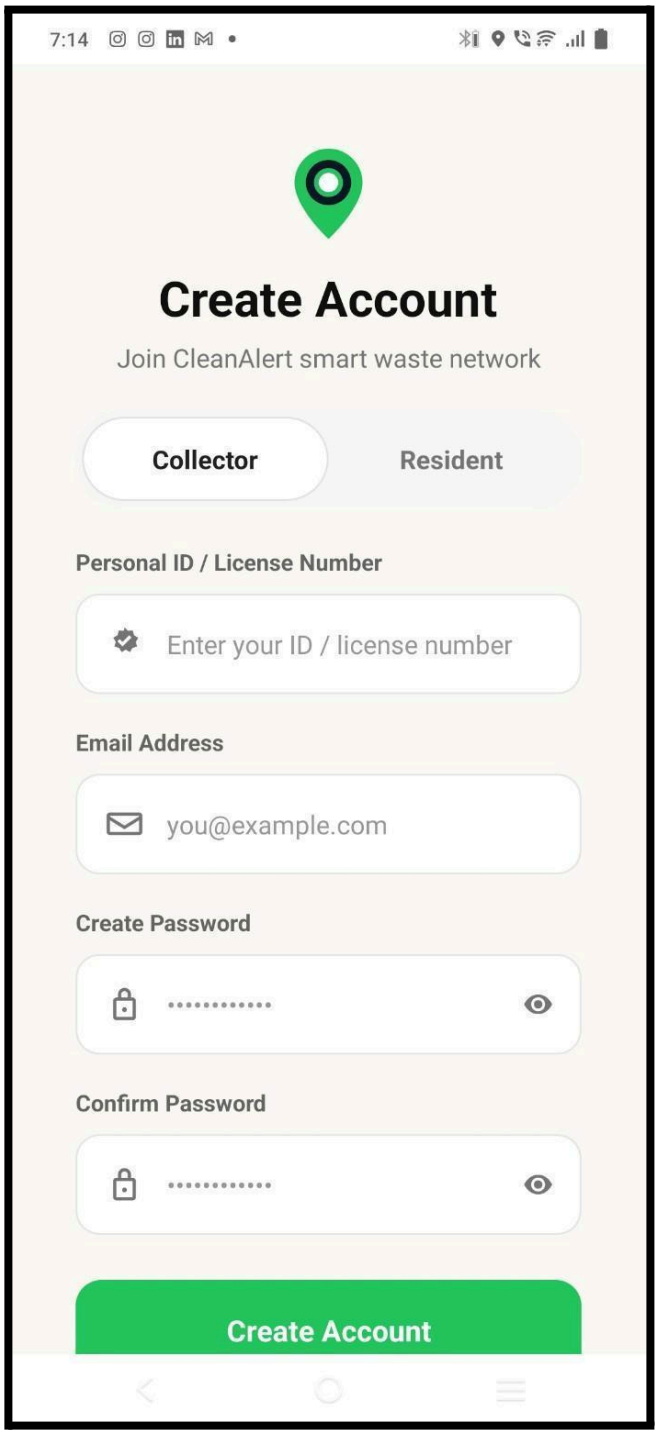



*Figure 7. Collector account registration screen*

Description: The collector registration screen includes a role selector and a personal ID or licence-number field in addition to email and password fields. The extra identifier is intended to support collector onboarding and later verification. The presence of the field alone does not verify the collector's identity; an administrator review or a separate verification process is required before granting operational access.

### 4.2.6 Submission Verification and Reward Management

The submission review module processes waste reports stored in the application backend.
Each submission record should contain a unique identifier, user ID, waste category, image reference, timestamp, and review status.
Photographic evidence is uploaded to cloud storage, while the associated metadata is maintained in the database.
The administrator retrieves pending submissions and evaluates them against the configured validation criteria.
After verification, the submission status is updated to either approved or rejected.
Status changes should be persisted in the backend to maintain consistency across user sessions.
Reward eligibility is determined by the submission status and the configured points policy.
The reward-processing logic should verify the current submission state before modifying the user's points balance.
Duplicate reward allocation should be prevented by associating each reward transaction with a unique submission identifier.
Reward balance updates should use an atomic database operation or transaction where supported.
During redemption, the backend must validate the user's available balance before confirming the transaction.
The system should record redemption details, including the reward identifier, points deducted, and transaction timestamp.
Failed operations should return an appropriate error response without partially updating related records.
These controls improve data integrity and reduce the risk of duplicate transactions.

## Security and Data Integrity

Role-based authorisation should be enforced through backend security rules rather than relying exclusively on interface visibility.
Residents should only be permitted to access or modify records authorised for their accounts.
Administrative operations, including approval and route assignment, should require verified administrative privileges.
Uploaded files should be validated for permitted MIME types and maximum file size before processing.
Firebase Authentication should establish user identity, while Firestore and Storage security rules should enforce resource-level access permissions.
Location and camera permissions should be requested at runtime only when the corresponding functionality is required.
Integration testing should verify submission status transitions, concurrent reward requests, redemption validation, and unauthorised access attempts.

# CHAPTER 5
# CONCLUSION AND FUTURE WORK

## 5.1 CONCLUSION

CleanAlert is a mobile-based smart waste management project intended to improve communication between residents, waste collectors, and administrators. The system brings together resident photo submissions, collector route visibility, administrator controls, and a community reward concept. The supplied application screenshots show the main account, administrator, and route-assignment interfaces.

The project provides a foundation for a more coordinated waste collection workflow. Its practical value depends on reliable backend integration, secure role permissions, accurate route data, dependable notification delivery, and a transparent reward-verification process. The interface should therefore be followed by systematic end-to-end testing and, where possible, a small field pilot.

## 5.2 FUTURE WORK

- Implement geofenced notifications when a collection vehicle approaches a locality.
- Add reliable live location updates with battery, privacy, and permission safeguards.
- Integrate route optimisation using stops, distance, traffic, and collection priorities.
- Add multilingual support for residents and collection workers.
- Provide offline-safe submission queues and retry handling.
- Introduce locality-level leaderboards and verified participation streaks.
- Build analytics for completed pickups, missed collections, response time, and submission volume.
- Run a field pilot and evaluate user participation and collection awareness with documented measurements.

## 5.3 TESTING AND RESULTS

Additional test coverage

The test plan should also cover invalid email formats, incorrect passwords, password-reset requests for unknown accounts, cancelled photo selection, unsupported image files, and uploads interrupted by a network failure. Role-access tests should confirm that residents cannot change route assignments or approve submissions, and that collectors cannot modify reward definitions or administer unrelated accounts. Reward tests should verify that an approved submission credits points only once, that rejected submissions do not earn points, and that redemption fails when the available balance is insufficient. Record the device model, Android version, build version, test date, network conditions, and observed result for every completed test. At present, the report identifies these as validation activities; it does not claim that the pending tests have passed.

The screenshots supplied for the report demonstrate interface structure, including account creation, sign-in, administrator overview, user/collector listing, and route assignment. Screenshots alone do not prove that authentication, data persistence, route updates, push notifications, or rewards operate correctly.

| TEST ID | TEST CASE | EXPECTED RESULT | STATUS |
|---|---|---|---|

| | | | |
|---|---|---|---|
| TC-01 | Resident registration | Account is created and resident workflow opens | Pending validation |
| TC-02 | Collector registration | Collector account is created with required details | Pending validation |
| TC-03 | Submit a waste photograph | Submission and image reference are stored | Pending validation |
| TC-04 | Review a submission | Admin can approve or reject and status changes | Pending validation |
| TC-05 | Award points | Approved submission credits points once | Pending validation |
| TC-06 | Assign collector route | Assigned route appears for the selected collector | Pending validation |
| TC-07 | Map/location permission | Map behaviour and permission handling are correct | Pending validation |
| TC-08 | Push notification | Configured event delivers notification to intended user | Pending validation |
| TC-09 | Unauthorized admin action | Backend denies the action | Pending validation |
| TC-10 | Poor network during upload | Clear error and safe retry without duplicate records | Pending validation |

No verified performance measurements were supplied with the screenshots. For the final report, record observed test results, test device and Android version, network conditions, number of trials, and any failures. Do not report a reduction in collection delays or a notification latency value unless it has been measured.

## 5.4 SOURCE CODE

### (MainActivity.java) - Main Application Logic (Android, Java):

```
package com.futureseed.cleanalert;

import android.Manifest;
import android.app.Activity;
import android.content.Intent;

import com.google.android.gms.auth.api.signin.GoogleSignIn;
import com.google.android.gms.auth.api.signin.GoogleSignInAccount;
import com.google.android.gms.auth.api.signin.GoogleSignInClient;
import com.google.android.gms.auth.api.signin.GoogleSignInOptions;
import com.google.android.gms.common.api.ApiException;
import com.google.android.gms.tasks.Task;
import com.google.firebase.auth.AuthCredential;
import com.google.firebase.auth.EmailAuthProvider;
import com.google.firebase.auth.UserInfo;
import com.google.firebase.firestore.AggregateField;
```

```
import com.google.firebase.firestore.AggregateSource;
import com.google.firebase.messaging.FirebaseMessaging;
import android.os.Build;
import com.google.firebase.auth.GoogleAuthProvider;
import android.content.pm.PackageManager;
import android.graphics.Bitmap;
import android.graphics.Color;
import android.net.Uri;
import android.os.Bundle;
import android.provider.MediaStore;
import android.text.InputType;
import android.view.Gravity;
import android.view.View;
import android.widget.EditText;
import android.widget.FrameLayout;
import android.widget.LinearLayout;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;

import android.location.Address;
import android.location.Geocoder;
import android.location.Location;

import com.google.android.gms.location.FusedLocationProviderClient;
import com.google.android.gms.location.LocationCallback;
import com.google.android.gms.location.LocationRequest;
import com.google.android.gms.location.LocationResult;
import com.google.android.gms.location.LocationServices;
import com.google.firebase.firestore.ListenerRegistration;
import com.google.android.gms.maps.CameraUpdateFactory;
import com.google.android.gms.maps.GoogleMap;
import com.google.android.gms.maps.MapView;
import com.google.android.gms.maps.OnMapReadyCallback;
import com.google.android.gms.maps.model.BitmapDescriptorFactory;
import com.google.android.gms.maps.model.LatLng;
import com.google.android.gms.maps.model.MarkerOptions;
import com.google.firebase.FirebaseApp;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.DocumentReference;
import com.google.firebase.firestore.DocumentSnapshot;
import com.google.firebase.firestore.FirebaseFirestore;
import com.google.firebase.firestore.Query;
import com.google.firebase.firestore.QueryDocumentSnapshot;
import com.google.firebase.firestore.SetOptions;
import com.google.firebase.firestore.Transaction;
import com.google.firebase.storage.FirebaseStorage;
import com.google.firebase.storage.StorageReference;

import com.google.firebase.storage.UploadTask;

import java.io.ByteArrayOutputStream;
import java.text.SimpleDateFormat;
import java.util.Calendar;
import java.util.Date;
import java.util.HashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;

public class MainActivity extends Activity implements OnMapReadyCallback {
    private static final int LOCATION_PERMISSION_REQUEST = 42;
```

```
    private static final int CAMERA_PERMISSION_REQUEST = 43;
    private static final int GALLERY_REQUEST = 700;
    private static final int CAMERA_REQUEST = 701;
    private static final int GOOGLE_SIGN_IN_REQUEST = 702;
    private GoogleSignInClient googleSignInClient;
    private static final String ADMIN_EMAIL = "admin@mc.com";
    private static final int NOTIFICATION_PERMISSION_REQUEST = 44;
    private static final SimpleDateFormat DAY_FORMAT =
            new SimpleDateFormat("yyyy-MM-dd", Locale.US);

    private String leaderboardScope = "locality";
    private String rewardsTab = "active";

    private int currentLayout = R.layout.activity_login;
    private FirebaseAuth auth;
    private FirebaseFirestore db;
    private String firebaseInitError;
    private Uri selectedPhotoUri;
    private Bitmap capturedPhotoBitmap;
    private String selectedWasteType = "Dry Recyclable";
    private Double capturedLat;
    private Double capturedLng;
    private String capturedAddress;
    private FusedLocationProviderClient fusedLocationClient;
    private String meName, meEmail, meRole, meLocality;
    private LatLng selectedTarget;
    private LocationCallback broadcastCallback;
    private boolean broadcastingLocation = false;
    private ListenerRegistration liveCollectorsListener;
    private final Map<String, com.google.android.gms.maps.model.Marker>
liveCollectorMarkers = new HashMap<>();
    private boolean waitingForCameraPermission;
    private MapView mapView;
    private GoogleMap googleMap;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        try {
            FirebaseApp firebaseApp = FirebaseApp.initializeApp(this);
            if (firebaseApp != null || !FirebaseApp.getApps(this).isEmpty()) {
                auth = FirebaseAuth.getInstance();
                db = FirebaseFirestore.getInstance();
            } else {
                firebaseInitError = "Firebase configuration is missing";
            }
        } catch (Exception error) {
            firebaseInitError = "Firebase could not initialize";
        }

        fusedLocationClient = LocationServices.getFusedLocationProviderClient(this);

        try {
            GoogleSignInOptions gso = new
GoogleSignInOptions.Builder(GoogleSignInOptions.DEFAULT_SIGN_IN)
                    .requestIdToken(getString(R.string.default_web_client_id))
                    .requestEmail()
                    .build();
            googleSignInClient = GoogleSignIn.getClient(this, gso);
        } catch (Exception error) {
            // default_web_client_id only exists once Google sign-in is enabled in
Firebase
            // Console and google-services.json has been re-downloaded. Button stays
inert
```

```
            // (with a clear toast) until then instead of crashing the app.
            googleSignInClient = null;
        }

        show(R.layout.activity_login);
        if (auth != null && db != null && auth.getCurrentUser() != null) {
            routeUser(auth.getCurrentUser());
        }
    }

    private void show(int layout) {
        if (currentLayout == R.layout.collector_map) teardownMap();
        currentLayout = layout;
        setContentView(layout);
        wireCurrentScreen();
        if (layout == R.layout.collector_map) setupLiveMap();
        if (layout == R.layout.collector_requests) loadCollectorRequests();
        if (layout == R.layout.admin_submissions) loadAdminSubmissions();
        if (layout == R.layout.resident_home) loadResidentHome();
        if (layout == R.layout.resident_points_streak) loadPointsStreak();
        if (layout == R.layout.resident_leaderboard) loadLeaderboard();
        if (layout == R.layout.resident_my_rewards) loadMyRewards();
        if (layout == R.layout.collector_dashboard) loadCollectorDashboard();
        if (layout == R.layout.admin_dashboard) loadAdminDashboard();
        if (layout == R.layout.resident_submit_waste) captureLocationForSubmission();
        if (layout == R.layout.resident_past_submissions) loadPastSubmissions("all");
        if (layout == R.layout.collector_profile) loadCollectorProfile();
        if (layout == R.layout.collection_history) loadCollectionHistory(null);
        if (layout == R.layout.resident_rewards) loadRewardsCatalog();
        if (layout == R.layout.resident_profile) loadResidentProfile();
        if (layout == R.layout.resident_collection_alert) loadCollectionAlert();
        applyIdentity();
        if (layout == R.layout.activity_login) {
            View apple = findViewById(R.id.btnApple);
            if (apple != null) apple.setVisibility(View.GONE);
        }
    }

    private void wireCurrentScreen() {
        link(R.id.tvForgotPassword, () -> show(R.layout.activity_forgot_password));
        link(R.id.tvCreateAccount, () -> show(R.layout.activity_register_resident));
        link(R.id.tvLogin, () -> show(R.layout.activity_login));
        link(R.id.tvAlreadyAccount, () -> show(R.layout.activity_login));
        link(R.id.btnSignIn, this::signIn);
        link(R.id.btnEditProfile, () -> showEditProfile(R.layout.resident_profile));
        link(R.id.btnNotifications, () ->
showNotificationSettings(R.layout.resident_profile));
        link(R.id.btnSettings, () -> showSettings(R.layout.resident_profile));
        link(R.id.btnRemarkFeedback, () -> showFeedback(R.layout.collector_profile));
        link(R.id.btnChangePassword, () ->
showChangePassword(R.layout.collector_profile));
        link(R.id.btnShiftSettings, () -> showSettings(R.layout.collector_profile));
        link(R.id.btnAdminUsers, this::showAdminUsers);
        link(R.id.btnAdminAssignRoutes, this::showAdminAssignRoutes);
        link(R.id.btnAdminRewards, this::showAdminRewards);
        link(R.id.btnAdminReports, this::showAdminReports);
        link(R.id.btnGoogle, this::startGoogleSignIn);
        link(R.id.btnCreateAccount, this::createAccount);
        link(R.id.btnSubmitWaste, () -> show(R.layout.resident_submit_waste));
        link(R.id.btnPickPhoto, this::pickPhoto);
        link(R.id.btnTakePhoto, this::takePhoto);
        link(R.id.btnSubmitWasteFinal, this::submitWaste);
        link(R.id.catDry, () -> setWasteCategory("Dry Recyclable"));
        link(R.id.catOrganic, () -> setWasteCategory("Organic (Wet)"));
        link(R.id.catEwaste, () -> setWasteCategory("E-Waste"));
```

```
        link(R.id.catOther, () -> setWasteCategory("Other"));
        link(R.id.filterAll, () -> loadPastSubmissions("all"));
        link(R.id.filterOrganic, () -> loadPastSubmissions("organic"));
        wireCollectionHistorySearch();
        link(R.id.btnHome, () -> show(R.layout.resident_home));
        link(R.id.tvCollector, () -> show(R.layout.activity_register_collector));
        link(R.id.tvResident, () -> show(R.layout.activity_register_resident));
        link(R.id.btnSendResetLink, this::sendPasswordReset);
        link(R.id.btnBackToLogin, () -> show(R.layout.activity_login));
        link(R.id.ivBack, () -> show(R.layout.activity_login));
        link(R.id.btnBack, this::goBack);

        link(R.id.navDashboard, () -> show(R.layout.collector_dashboard));
        link(R.id.navMap, () -> show(R.layout.collector_map));
        link(R.id.navHistory, () -> show(R.layout.collection_history));
        link(R.id.navProfile, () -> show(R.layout.collector_profile));
        link(R.id.btnToggleStatus, this::toggleCollectorStatus);



        link(R.id.navHome, () -> show(R.layout.resident_home));
        link(R.id.navResHistory, () -> show(R.layout.resident_past_submissions));
        link(R.id.navRewards, () -> show(R.layout.resident_rewards));
        link(R.id.navResProfile, () -> show(R.layout.resident_profile));
        link(R.id.openPointsStreak, () -> show(R.layout.resident_points_streak));
        link(R.id.tvHomeStreak, () -> show(R.layout.resident_leaderboard));
        link(R.id.btnNavigate, this::openNavigation);
        link(R.id.btnLogout, this::logout);
        link(R.id.adminLogout, this::logout);
        link(R.id.btnAdminSubmissions, () -> { adminListMode = "submissions";
show(R.layout.admin_submissions); });
        link(R.id.btnAdminBack, () -> show(R.layout.admin_dashboard));
        link(R.id.btnAdminQueueLogout, this::logout);
        link(R.id.btnCollectorBack, () -> show(R.layout.collector_dashboard));
        link(R.id.btnViewDetails, () -> show(R.layout.collector_profile));

        link(R.id.btnLocality, () -> setLeaderboardScope("locality"));
        link(R.id.btnCity, () -> setLeaderboardScope("city"));
        link(R.id.tabActive, () -> setRewardsTab("active"));
        link(R.id.tabRedeemed, () -> setRewardsTab("redeemed"));
        link(R.id.btnViewAll, () -> show(R.layout.resident_my_rewards));
        link(R.id.btnDone, () -> show(R.layout.resident_my_rewards));
        link(R.id.btnViewMyRewards, () -> show(R.layout.resident_my_rewards));
        link(R.id.btnBackToRewards, () -> show(R.layout.resident_rewards));

        link(R.id.btnZoomIn, () -> { if (googleMap != null)
googleMap.animateCamera(CameraUpdateFactory.zoomIn()); });
        link(R.id.btnZoomOut, () -> { if (googleMap != null)
googleMap.animateCamera(CameraUpdateFactory.zoomOut()); });
        link(R.id.btnMyLocation, () -> enableMyLocation());
    }

    private void signIn() {
        if (auth == null) {
            toast(firebaseUnavailableMessage());
            return;
        }
        EditText email = findViewById(R.id.etEmail);
        EditText password = findViewById(R.id.etPassword);
        String emailText = email == null ? "" : email.getText().toString().trim();
        String passwordText = password == null ? "" : password.getText().toString();
        if (emailText.isEmpty() || passwordText.isEmpty()) {
            toast("Enter your email and password");
            return;
        }
```

```
        auth.signInWithEmailAndPassword(emailText, passwordText)
                .addOnSuccessListener(result -> routeUser(result.getUser()))
                .addOnFailureListener(error -> toast(error.getMessage() == null ? "Sign
in failed" : error.getMessage()));
    }

    private void createAccount() {
        if (auth == null || db == null) {
            toast(firebaseUnavailableMessage());
            return;
        }
        EditText email = findViewById(R.id.etEmail);
        EditText password = findViewById(R.id.etPassword);
        EditText confirm = findViewById(R.id.etConfirmPassword);
        String emailText = email == null ? "" : email.getText().toString().trim();
        String passwordText = password == null ? "" : password.getText().toString();
        String confirmText = confirm == null ? passwordText :
confirm.getText().toString();
        if (emailText.isEmpty() || passwordText.length() < 6) {
            toast("Use an email and a password with at least 6 characters");
            return;
        }
        if (!passwordText.equals(confirmText)) {
            toast("Passwords do not match");
            return;
        }
        String role = ADMIN_EMAIL.equalsIgnoreCase(emailText)
                ? "admin"
                : (currentLayout == R.layout.activity_register_collector ? "collector" :
"resident");
        auth.createUserWithEmailAndPassword(emailText, passwordText)
                .addOnSuccessListener(result -> {
                    FirebaseUser user = result.getUser();
                    if (user == null) return;
                    Map<String, Object> profile = new HashMap<>();
                    profile.put("uid", user.getUid());
                    profile.put("email", emailText);
                    profile.put("role", role);
                    profile.put("createdAt", System.currentTimeMillis());
                    String displayName = emailText.contains("@") ?
emailText.substring(0, emailText.indexOf('@')) : emailText;
                    if ("resident".equals(role)) {
                        profile.put("points", 0L);
                        profile.put("streak", 0L);
                        profile.put("name", displayName);
                    } else if ("collector".equals(role)) {
                        profile.put("status", "offline");
                        profile.put("name", displayName);
                    }
                    db.collection("users").document(user.getUid()).set(profile)
                            .addOnSuccessListener(done -> {
                                if ("resident".equals(role)) {
                                    Map<String, Object> board = new HashMap<>();
                                    board.put("name", displayName);
                                    board.put("points", 0L);
                                    board.put("streak", 0L);

db.collection("leaderboard").document(user.getUid()).set(board);
                                }
                                routeUser(user);
                            })
                            .addOnFailureListener(error -> toast("Account created, but
profile setup failed"));
                })
```

```
                .addOnFailureListener(error -> toast(error.getMessage() == null ?
"Account creation failed" : error.getMessage()));
    }
```

## REFERENCES


[1] D. D. Meshram, R. R. Sahu, S. V. Pathak, K. P. Fukte, A. P. Darodkar, G. S. Ramtekkar, and M. S. Meshram, “Smart civic issue reporting, resolution & waste recycle reward system,” Int. J. Eng. Res. Technol., in press.

[2] S. Prakash, R. Chaurasia, and P. K. Chaubey, “Smart civic issue reporting system with hybrid AI classification and geospatial analytics for efficient urban governance,” in Int. Conf. Proc., in press.

[3] V. Naveen, I. Umesh, M. Raghavender, B. Vaishnavi, G. Devi, and B. Venkataramana, “A theoretical framework for AI-assisted civic issue reporting and validation in smart cities,” in Int. Conf. Proc., in press.

[4] J. Rosenlund, M. Helmefalk, S. Stenfelt, and A. Palmquist, “Leveling up the recycling experience: Gamification of recycling through an innovative recycling station,” Resour. Conserv. Recycl., in press.

[5] S. Satheesh, V. Pradhan, K. Shukla, and Y. Patil, “A total interpretive structural modeling (TISM) framework for gamification in solid waste management and its influence on quality of life,” J. Clean. Prod., in press.

[6] Google, “FusedLocationProviderClient,” Android Developers Documentation. [Online]. Available: https://developer.android.com

[7] Google, “Maps SDK for Android,” Google Maps Platform Documentation. [Online]. Available: https://developers.google.com/maps

[8] Google LLC, “Firebase: Cloud Firestore, Cloud Messaging, Authentication and Cloud Storage,” Firebase Documentation. [Online]. Available: https://firebase.google.com/docs

SIMATS ENGINEERING

ENGINEER TO EXCEL

NBA Tier 1 | IET-UK | ABET Accreditation

QS I·GAUGE INDIAN UNIVERSITY RATING PLATINUM

NAAC A++ GRADE

nirf 45 RANK

INSTITUTION'S INNOVATION COUNCIL

MoE's INNOVATION CELL (GOVERNMENT OF INDIA)

IQAC

SUSTAINABLE DEVELOPMENT GOALS

# CERTIFICATE OF PARTICIPATION

This certificate is presented to

SOURABH KUMAR

VELS UNIVERSITY

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

CLEAN ALERT &SMART GPS-ENABLED REAL-TIME WASTE COLLECTION NOTIFICATION AND COMMUNITY REWARD SYSTEM

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
**Dr. B. Ramesh**

SIMATS ENGINEERING

ENGINEER TO EXCEL

NBA Tier 1 | IET-UK | ABET Accreditation

QS I·GAUGE INDIAN UNIVERSITY RATING PLATINUM

NAAC A++ GRADE

nirf 45 RANK

INSTITUTION'S INNOVATION COUNCIL

MoE's INNOVATION CELL (GOVERNMENT OF INDIA)

IQAC

SUSTAINABLE DEVELOPMENT GOALS

# CERTIFICATE OF PARTICIPATION

This certificate is presented to

DEVENDRA SINGH

VELS UNIVERSITY

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

CLEAN ALERT & SMART GPS-ENABLED REAL-TIME WASTE COLLECTION NOTIFICATION AND COMMUNITY REWARD SYSTEM

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
**Dr. B. Ramesh**

SIMATS ENGINEERING

ENGINEER TO EXCEL

NBA Tier 1 | IET-UK | ABET Accreditation

QS I-GAUGE INDIAN UNIVERSITY RATING PLATINUM | NAAC A++ GRADE | nirf 45 RANK

INSTITUTION'S INNOVATION COUNCIL | MoE's INNOVATION CELL (GOVERNMENT OF INDIA) | IQAC | SUSTAINABLE DEVELOPMENT GOALS

# CERTIFICATE OF EXCELLENCE

This certificate is presented to

SOURABH KUMAR

VELS UNIVERSITY

for securing **Best Paper Award** in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** for the Research Paper titled

Clean Alert : A smart GPS-Enabled Real-time waste collection Notification and community Reward System for Effiient and Responsible urban waste management

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

N. Bly

Conference Chair
ICAI-CME 2026

B. Ramesh

Principal
**Dr. B. Ramesh**

ENGINEER TO EXCEL

# SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

QS I·GAUGE INDIAN UNIVERSITY RATING PLATINUM

NAAC A++ GRADE

nirf 45 RANK

INSTITUTION'S INNOVATION COUNCIL

MoE's INNOVATION CELL (GOVERNMENT OF INDIA)

IQAC

SUSTAINABLE DEVELOPMENT GOALS

# CERTIFICATE OF EXCELLENCE

This certificate is presented to

DEVENDRA SINGH

VELS UNIVERSITY

for securing **Best Paper Award** in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** for the Research Paper titled

Clean Alert : A Smart GPS - Enabled Real-Time Waste Collection, Notification & Community Reward System for Efficient & Responsible Urban Waste Management.

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai.

Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
**Dr. B. Ramesh**